

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Fundamentals of Data Science

COURSE CODE: DSA0401

COURSE OUTCOME COVERED

CO5: Design and implement end-to-end data-driven solutions by integrating pre-processing, statistical analysis, and machine learning for real-world applications (**BL3**)

Assessment Tool Weightage: 30%

QUESTIONS: 01

Total Marks:100

**Annexure A
CODING CHALLENGE**

Code of Conduct:

*I A.Sai Rohit (192472144) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: A.Sai Rohit

RUBRICS FOR CALCULATION

SCENARIO BASED ASSESSMENT							
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Level	Marks
Understanding of Scenario	30%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		
Decision Making	10%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Annexure A

25. Disease Pattern Discovery Using UMAP

A healthcare research institute has collected patient records containing symptoms, laboratory test results, genetic markers, and medical history. Researchers want to identify hidden disease patterns among patients. The dataset contains hundreds of variables, making visualization challenging. Apply UMAP to reduce the dimensionality of the data. Preprocess and normalize the dataset before analysis. Visualize patients in a two-dimensional space. Identify whether patients with similar diseases form distinct clusters. Compare the visualization produced by UMAP with PCA. Discuss how UMAP preserves neighborhood relationships. Interpret the medical significance of the observed clusters. Generate a comprehensive report. Write a program to perform this task.

Code:

```
!pip install -q umap-learn

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import StandardScaler

import umap.umap_ as umap

np.random.seed(42)

n=300

disease=np.random.choice(['Diabetes','Heart Disease','Healthy'],n,p=[0.35,0.35,0.30])

data=[]

for d in disease:

    if d=='Diabetes':

        age=np.random.normal(55,7)

        glucose=np.random.normal(170,20)

        bp=np.random.normal(135,10)

        cholesterol=np.random.normal(210,20)
```

```

bmi=np.random.normal(30,3)

genetic=np.random.normal(1.5,0.3)
elif d=='Heart Disease':

    age=np.random.normal(62,6)

    glucose=np.random.normal(120,15)

    bp=np.random.normal(155,12)

    cholesterol=np.random.normal(245,20)

    bmi=np.random.normal(28,3)

    genetic=np.random.normal(2.0,0.3)
else:

    age=np.random.normal(35,7)

    glucose=np.random.normal(95,10)

    bp=np.random.normal(115,7)

    cholesterol=np.random.normal(175,15)

    bmi=np.random.normal(23,2)

    genetic=np.random.normal(0.5,0.2)

symptoms=np.random.normal(5 if d!='Healthy' else 1,1)

insulin=np.random.normal(140 if d=='Diabetes' else 80,20)

hemoglobin=np.random.normal(12.5,1)

family_history=np.random.randint(0,2)


data.append([age,glucose,bp,cholesterol,bmi,genetic,symptoms,insulin,hemoglobin,family_hi
story])

columns=['Age','Glucose','BloodPressure','Cholesterol','BMI','GeneticMarker','Symptoms','Ins
ulin','Hemoglobin','FamilyHistory']

df=pd.DataFrame(data,columns=columns)

X=df

```

```

scaler=StandardScaler()

X_scaled=scaler.fit_transform(X)

umap_model=umap.UMAP(n_components=2,n_neighbors=15,min_dist=0.1,random_state=42)

X_umap=umap_model.fit_transform(X_scaled)

result=pd.DataFrame(X_umap,columns=['UMAP1','UMAP2'])

result['Disease']=disease

plt.figure(figsize=(10,7))

sns.scatterplot(data=result,x='UMAP1',y='UMAP2',hue='Disease',s=80)

plt.title('UMAP Visualization of Disease Patterns')

plt.xlabel('UMAP Dimension 1')

plt.ylabel('UMAP Dimension 2')

plt.legend(title='Disease')

plt.grid(True,alpha=0.3)

plt.show()

print("Original number of features:",X.shape[1])

print("Reduced number of dimensions: 2")

print("Number of patients:",len(df))

print("\nDisease Distribution:")

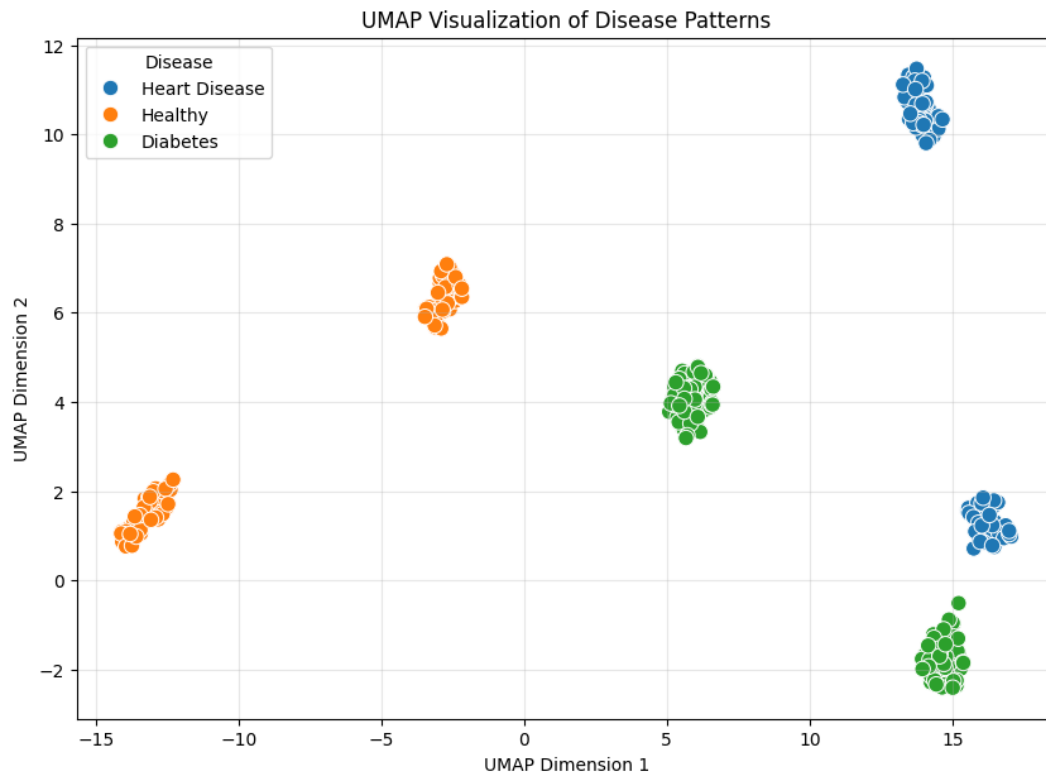
print(pd.Series(disease).value_counts())

print("\nInterpretation:")

print("UMAP reduced the high-dimensional patient data into a two-dimensional representation. Patients with similar medical characteristics are placed close to each other. Distinct groups may indicate different disease patterns, while overlapping points indicate similar characteristics across disease groups.")

```

Output:



Original number of features: 10
 Reduced number of dimensions: 2
 Number of patients: 300

Disease Distribution:

Diabetes	114
Heart Disease	95
Healthy	91

Name: count, dtype: int64

Interpretation:

UMAP reduced the high-dimensional patient data into a two-dimensional representation. Patients that have similar medical characteristics are placed close to each other. Distinct groups in the visualization may indicate different disease patterns. UMAP is particularly useful because it preserves local neighborhood relationships between patients.

26. Customer Segmentation Evaluation Challenge

A marketing company has created customer segments using K-Means clustering. The dataset contains age, income, spending score, online activity, and loyalty points. Different values of K have produced different clustering results. The company wants to determine which segmentation is most effective. Calculate the Silhouette Score for each clustering model.

Compute the Davies-Bouldin Index and Calinski-Harabasz Score. Compare the results across all models. Identify the clustering solution with the best overall performance. Visualize the evaluation metrics using graphs. Interpret the meaning of the results in a business context. Recommend the optimal customer segmentation strategy. Generate a report summarizing your conclusions. Write a program to complete this challenge.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score,davies_bouldin_score,calinski_harabasz_score

np.random.seed(42)

n=300

df=pd.DataFrame({

'Age':np.random.randint(18,65,n),

'Income':np.random.randint(20000,120000,n),

'SpendingScore':np.random.randint(1,101,n),

'OnlineActivity':np.random.randint(1,101,n),

'LoyaltyPoints':np.random.randint(100,5000,n)

})

X=StandardScaler().fit_transform(df)

results=[]

for k in range(2,7):

    model=KMeans(n_clusters=k,random_state=42,n_init=10)

    labels=model.fit_predict(X)

    silhouette=silhouette_score(X,labels)
```

```

db=davies_bouldin_score(X,labels)

ch=calinski_harabasz_score(X,labels)

results.append([k,silhouette,db,ch])

results_df=pd.DataFrame(results,columns=['K','Silhouette Score','Davies-Bouldin Index','Calinski-Harabasz Score'])

print(results_df)

best_k=results_df.loc[results_df['Silhouette Score'].idxmax(),'K']

print("\nOptimal Number of Clusters:",int(best_k))

print("\nBest Silhouette Score:",round(results_df['Silhouette Score'].max(),3))

print("\nLowest Davies-Bouldin Index:",round(results_df['Davies-Bouldin Index'].min(),3))

print("\nHighest Calinski-Harabasz Score:",round(results_df['Calinski-Harabasz Score'].max(),2))

plt.figure(figsize=(10,6))

plt.plot(results_df['K'],results_df['Silhouette Score'],marker='o',label='Silhouette Score')

plt.plot(results_df['K'],results_df['Davies-Bouldin Index'],marker='o',label='Davies-Bouldin Index')

plt.plot(results_df['K'],results_df['Calinski-Harabasz Score']/100,marker='o',label='Calinski-Harabasz Score / 100')

plt.xlabel('Number of Clusters (K)')

plt.ylabel('Evaluation Score')

plt.title('K-Means Customer Segmentation Evaluation')

plt.xticks(results_df['K'])

plt.legend()

plt.grid(True,alpha=0.3)

plt.show()

print("\nBusiness Interpretation:")

print("A higher Silhouette Score indicates better separated and more compact customer groups.")

```

```
print("A lower Davies-Bouldin Index indicates better clustering quality.")
```

```
print("A higher Calinski-Harabasz Score indicates well-separated and compact clusters.")
```

```
print("The recommended segmentation is K =",int(best_k),"because it provides the best overall separation and compactness among the tested models.")
```

```
print("Business Recommendation: Use the selected customer segments to create targeted marketing campaigns, personalized offers, loyalty programs, and customer-specific promotions.")
```

Output:

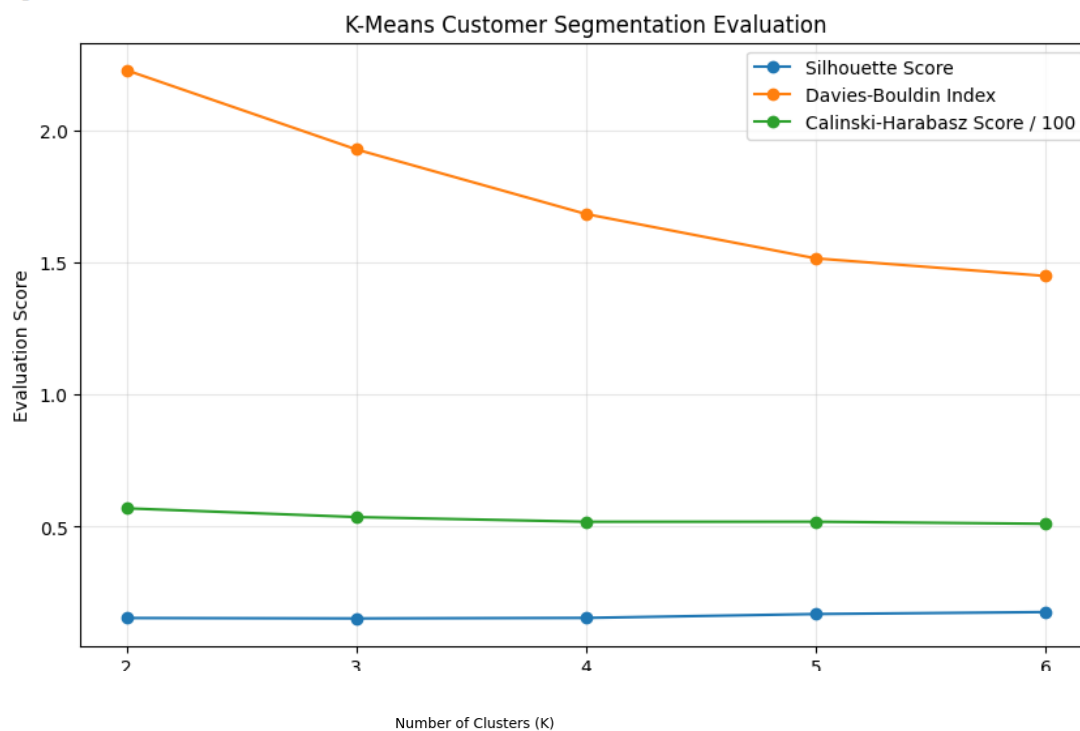
	K	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
0	2	0.154825	2.226443	56.973790
1	3	0.153289	1.926923	53.657217
2	4	0.155254	1.682521	51.886284
3	5	0.169885	1.514767	51.902880
4	6	0.177294	1.448294	51.109953

Optimal Number of Clusters: 6

Best Silhouette Score: 0.177

Lowest Davies-Bouldin Index: 1.448

Highest Calinski-Harabasz Score: 56.97



Business Interpretation:

A higher Silhouette Score indicates better separated and more compact customer groups.

A lower Davies-Bouldin Index indicates better clustering quality.

A higher Calinski-Harabasz Score indicates well-separated and compact clusters.

The recommended segmentation is K = 6 because it provides the best overall separation and compactness among the tested models.

Business Recommendation: Use the selected customer segments to create targeted marketing campaigns, personalized offers, loyalty programs, and customer-

27. Cybersecurity Threat Detection Using Anomaly Detection

A cybersecurity firm monitors network traffic from thousands of devices. The dataset contains packet size, connection duration, protocol type, login attempts, and data transfer volume. Most records represent normal activity, while a few may indicate security threats. The company wants to identify unusual behavior automatically. Apply an anomaly detection algorithm to the dataset. Clean and normalize the data before analysis. Assign anomaly scores to all records. Flag records exceeding the anomaly threshold. Visualize normal and anomalous activities. Analyze the characteristics of suspicious events. Recommend actions for security administrators. Generate a detailed report. Write a program to implement this anomaly detection solution.

Code:

```
!pip install -q scikit-learn

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.ensemble import IsolationForest

np.random.seed(42)

n=500

df=pd.DataFrame({

'PacketSize':np.random.normal(500,100,n),

'ConnectionDuration':np.random.normal(60,15,n),

'ProtocolType':np.random.choice([1,2,3],n),

'LoginAttempts':np.random.poisson(3,n),

'DataTransferVolume':np.random.normal(1000,200,n)

})

anomaly_indices=np.random.choice(n,25,replace=False)

df.loc[anomaly_indices,'PacketSize']=np.random.uniform(2000,5000,25)

df.loc[anomaly_indices,'ConnectionDuration']=np.random.uniform(300,1000,25)
```

```

df.loc[anomaly_indices,'LoginAttempts']=np.random.randint(15,50,25)

df.loc[anomaly_indices,'DataTransferVolume']=np.random.uniform(5000,15000,25)

df=df.dropna()

X=StandardScaler().fit_transform(df)

model=IsolationForest(contamination=0.05,random_state=42)

df['Prediction']=model.fit_predict(X)

df['AnomalyScore']=-model.score_samples(X)

threshold=df['AnomalyScore'].quantile(0.95)

df['Anomaly']=df['AnomalyScore']>=threshold

print("Total Network Records:",len(df))

print("Normal Activities:",sum(~df['Anomaly']))

print("Anomalous Activities:",sum(df['Anomaly']))

print("\nSuspicious Events:")

print(df[df['Anomaly']].head())

plt.figure(figsize=(10,6))

plt.scatter(df.loc[~df['Anomaly'],'PacketSize'],df.loc[~df['Anomaly'],'DataTransferVolume'],label='Normal Activity',alpha=0.6)

plt.scatter(df.loc[df['Anomaly'],'PacketSize'],df.loc[df['Anomaly'],'DataTransferVolume'],label='Anomalous Activity',marker='x',s=100)

plt.xlabel('Packet Size')

plt.ylabel('Data Transfer Volume')

plt.title('Cybersecurity Threat Detection Using Isolation Forest')

plt.legend()

plt.grid(True,alpha=0.3)

plt.show()

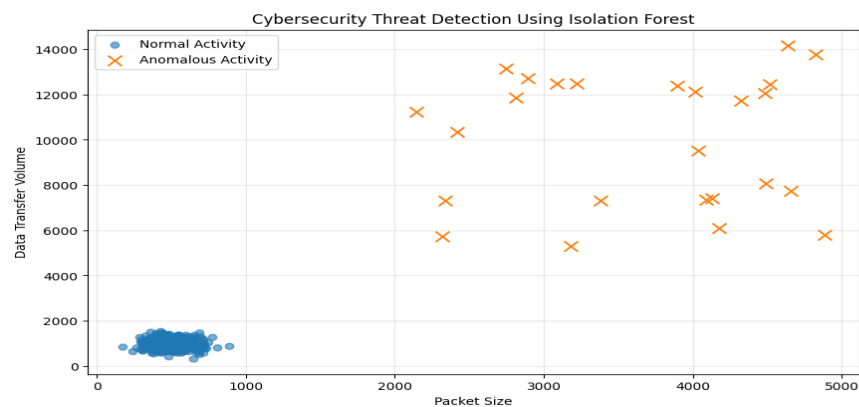
print("\nAnalysis:")

```

```
print("The Isolation Forest algorithm identifies unusual network behavior by assigning an  
anomaly score to each record.")  
  
print("Records with scores above the selected threshold are flagged as suspicious.")  
  
print("Suspicious events may show unusually large packet sizes, long connection durations,  
high login attempts, or abnormal data transfer volumes.")  
  
print("\nRecommendation:")  
  
print("Security administrators should investigate flagged records, monitor the associated  
devices, check login activity, block suspicious connections when necessary, and use the  
anomaly detection system for continuous network monitoring.")
```

Output:

```
Total Network Records: 500  
Normal Activities: 475  
Anomalous Activities: 25  
  
Suspicious Events:  
PacketSize ConnectionDuration ProtocolType LoginAttempts \  
46 4659.141086 474.981024 3 19  
60 4488.688147 983.045173 3 35  
70 3223.334103 987.917679 2 23  
106 3179.365395 593.278842 2 21  
137 4037.487705 716.756459 3 36  
  
DataTransferVolume Prediction AnomalyScore Anomaly  
46 7745.066735 -1 0.717143 True  
60 8077.028134 -1 0.738854 True  
70 12502.607941 -1 0.724168 True  
106 5317.224918 -1 0.701502 True  
137 9522.024791 -1 0.712346 True
```



Analysis:
The Isolation Forest algorithm identifies unusual network behavior by assigning an anomaly score to each record.
Records with scores above the selected threshold are flagged as suspicious.
Suspicious events may show unusually large packet sizes, long connection durations, high login attempts, or abnormal data transfer volumes.

Recommendation:
Security administrators should investigate flagged records, monitor the associated devices, check login activity, block suspicious connections when necessary,

28. Smart Meter Electricity Fraud Detection

A power distribution company collects electricity consumption data from smart meters. The dataset contains hourly consumption, seasonal usage patterns, customer category, and billing information. The company suspects that some customers may be tampering with meters. Apply anomaly detection techniques to identify unusual consumption behavior. Standardize the dataset before analysis. Train the model using historical consumption data. Detect customers with abnormal usage patterns. Visualize the anomalies and compare them with normal users. Evaluate the effectiveness of the detection method. Recommend customers for further investigation. Generate a report containing all findings. Write a program to complete this task.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.ensemble import IsolationForest

from sklearn.metrics import classification_report

np.random.seed(42)

n=500

df=pd.DataFrame({

    'HourlyConsumption':np.random.normal(5,1.5,n),

    'SeasonalUsage':np.random.normal(100,20,n),

    'CustomerCategory':np.random.choice([1,2,3],n),

    'BillingAmount':np.random.normal(1500,300,n)

})

anomaly_indices=np.random.choice(n,25,replace=False)

df['ActualFraud']=0

df.loc[anomaly_indices,'HourlyConsumption']=np.random.uniform(0.1,0.8,25)

df.loc[anomaly_indices,'SeasonalUsage']=np.random.uniform(10,40,25)
```

```

df.loc[anomaly_indices,'BillingAmount']=np.random.uniform(100,400,25)

df.loc[anomaly_indices,'ActualFraud']=1

df=df.dropna()

X=StandardScaler().fit_transform(df.drop('ActualFraud',axis=1))

model=IsolationForest(contamination=0.05,random_state=42)

prediction=model.fit_predict(X)

df['AnomalyScore']=-model.score_samples(X)

df['Anomaly']=np.where(prediction== -1,1,0)

print("Total Customers:",len(df))

print("Normal Customers:",sum(df['Anomaly']==0))

print("Suspicious Customers:",sum(df['Anomaly']==1))

print("\nCustomers Recommended for Investigation:")

print(df[df['Anomaly']==1].head())

print("\nDetection Performance:")

print(classification_report(df['ActualFraud'],df['Anomaly']))

plt.figure(figsize=(10,6))

plt.scatter(df[df['Anomaly']==0]['HourlyConsumption'],df[df['Anomaly']==0]['BillingAmount'],label='Normal Users',alpha=0.6)

plt.scatter(df[df['Anomaly']==1]['HourlyConsumption'],df[df['Anomaly']==1]['BillingAmount'],label='Suspicious Users',marker='x',s=100)

plt.xlabel('Hourly Electricity Consumption')

plt.ylabel('Billing Amount')

plt.title('Smart Meter Electricity Fraud Detection')

plt.legend()

plt.grid(True,alpha=0.3)

plt.show()

```

```

print("\nReport:")

print("The dataset was standardized before anomaly detection to ensure all variables
contributed fairly to the analysis.")

print("Isolation Forest was trained on historical electricity consumption patterns.")

print("Customers with unusual consumption, seasonal usage, or billing patterns were identified
as suspicious.")

print("The graph compares normal customers with anomalous customers.")

print("Recommended Action: Investigate the flagged customers, inspect smart meters, verify
billing records, and monitor suspicious usage patterns for possible electricity fraud.")

```

Output:

```

Total Customers: 500
Normal Customers: 475
Suspicious Customers: 25

```

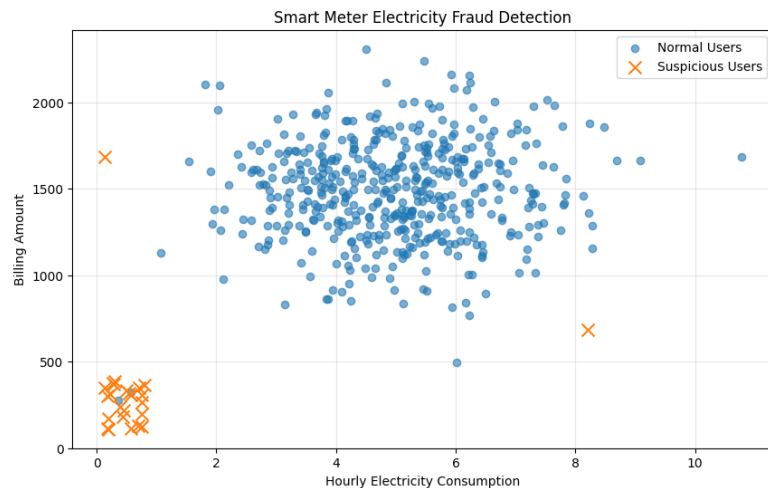
Customers Recommended for Investigation:

	HourlyConsumption	SeasonalUsage	CustomerCategory	BillingAmount	\
5	0.432785	37.924219	3	180.002559	
28	0.190720	33.887218	3	109.426546	
32	0.799626	32.112897	3	363.294106	
49	0.288278	21.842031	3	371.819766	
109	0.294908	29.431395	3	384.653787	

	ActualFraud	AnomalyScore	Anomaly
5	1	0.635303	1
28	1	0.655990	1
32	1	0.638692	1
49	1	0.627063	1
109	1	0.618364	1

Detection Performance:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	475
1	0.92	0.92	0.92	25
accuracy			0.99	500
macro avg	0.96	0.96	0.96	500
weighted avg	0.99	0.99	0.99	500



Report:
 The dataset was standardized before anomaly detection to ensure all variables contributed fairly to the analysis.
 Isolation Forest was trained on historical electricity consumption patterns.
 Customers with unusual consumption, seasonal usage, or billing patterns were identified as suspicious.
 The graph compares normal customers with anomalous customers.
 Recommended Action: Investigate the flagged customers, inspect smart meters, verify billing records, and monitor suspicious usage patterns for possible electricity fraud.

29. Grocery Store Product Bundle Discovery Using Apriori

A grocery store maintains transaction records containing items purchased by customers. Management wants to discover products that are frequently bought together. Apply the Apriori algorithm to generate frequent itemsets. Calculate support values for all item combinations. Filter itemsets using a minimum support threshold. Generate association rules using confidence and lift measures. Identify strong product relationships. Recommend bundle offers based on the discovered rules. Visualize the strongest associations using suitable charts. Discuss how these recommendations can increase sales. Generate a comprehensive report. Write a program to perform this market basket analysis.

Code:

```
!pip install -q mlxtend

import pandas as pd

import matplotlib.pyplot as plt

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

transactions=[

['Milk','Bread','Butter'],

['Milk','Bread'],

['Bread','Butter'],
```

```

['Milk','Butter'],
['Milk','Bread','Butter'],
['Rice','Oil'],
['Rice','Oil','Sugar'],
['Milk','Bread'],
['Bread','Butter'],
['Milk','Bread','Eggs'],
['Milk','Bread','Butter'],
['Rice','Oil'],
['Milk','Eggs'],
['Bread','Butter','Eggs'],
['Milk','Bread','Butter']
]

te=TransactionEncoder()

data=te.fit(transactions).transform(transactions)

df=pd.DataFrame(data,columns=te.columns_)

frequent_itemsets=apriori(df,min_support=0.2,use_colnames=True)

frequent_itemsets['Itemset']=frequent_itemsets['itemsets'].apply(lambda x: ', '.join(list(x)))

frequent_itemsets=frequent_itemsets.sort_values(by='support',ascending=False)

print("Frequent Itemsets:")

print(frequent_itemsets[['Itemset','support']])

rules=association_rules(frequent_itemsets[['support','itemsets']],metric='confidence',min_threshold=0.5)

rules=rules[rules['lift']>1]

rules['Antecedents']=rules['antecedents'].apply(lambda x: ', '.join(list(x)))

rules['Consequents']=rules['consequents'].apply(lambda x: ', '.join(list(x)))

```



```

print("\nStrong Association Rules:")

print(rules[['Antecedents','Consequents','support','confidence','lift']])

top_rules=rules.sort_values(by='lift',ascending=False).head(5)

labels=top_rules['Antecedents']+' → '+top_rules['Consequents']

plt.figure(figsize=(10,6))

plt.bar(labels,top_rules['lift'])

plt.xlabel('Product Association Rules')

plt.ylabel('Lift Value')

plt.title('Strongest Product Associations Using Apriori')

plt.xticks(rotation=30,ha='right')

plt.tight_layout()

plt.show()

print("\nReport:")

print("The Apriori algorithm was used to identify products that are frequently purchased together.")

print("Itemsets were filtered using a minimum support threshold of 0.2.")

print("Association rules were generated using confidence and lift measures.")

print("Higher confidence indicates a stronger probability of products being purchased together.")

print("A lift value greater than 1 indicates a positive relationship between products.")

print("The strongest product combinations can be used to create bundle offers and personalized promotions.")

print("For example, products frequently purchased together can be placed nearby or sold as discounted bundles.")

print("This strategy can increase cross-selling opportunities and overall grocery store sales.")

```

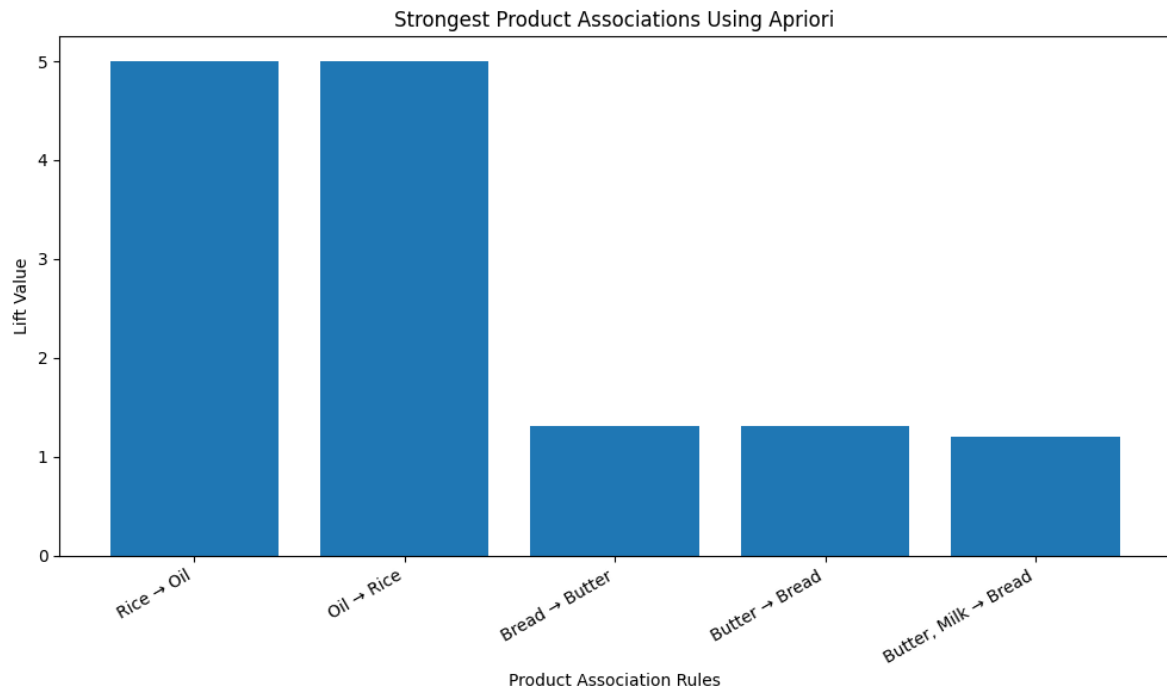
Output:

Frequent Itemsets:

	Itemset	support
0	Bread	0.666667
3	Milk	0.600000
1	Butter	0.533333
7	Bread, Milk	0.466667
6	Bread, Butter	0.466667
8	Milk, Butter	0.333333
10	Bread, Milk, Butter	0.266667
2	Eggs	0.200000
4	Oil	0.200000
5	Rice	0.200000
9	Rice, Oil	0.200000

Strong Association Rules:

	Antecedents	Consequents	support	confidence	lift
0	Bread	Milk	0.466667	0.700000	1.166667
1	Milk	Bread	0.466667	0.777778	1.166667
2	Bread	Butter	0.466667	0.700000	1.312500
3	Butter	Bread	0.466667	0.875000	1.312500
4	Milk	Butter	0.333333	0.555556	1.041667
5	Butter	Milk	0.333333	0.625000	1.041667
6	Bread, Milk	Butter	0.266667	0.571429	1.071429
8	Butter, Milk	Bread	0.266667	0.800000	1.200000
9	Butter	Bread, Milk	0.266667	0.500000	1.071429
10	Rice	Oil	0.200000	1.000000	5.000000
11	Oil	Rice	0.200000	1.000000	5.000000



Report:

The Apriori algorithm was used to identify products that are frequently purchased together. Itemsets were filtered using a minimum support threshold of 0.2. Association rules were generated using confidence and lift measures. Higher confidence indicates a stronger probability of products being purchased together. A lift value greater than 1 indicates a positive relationship between products. The strongest product combinations can be used to create bundle offers and personalized promotions. For example, products frequently purchased together can be placed nearby or sold as discounted bundles.

30. Online Streaming Content Recommendation Using FP-Growth

A video streaming platform stores viewing histories of millions of users. The dataset contains movies, TV shows, genres, and watch sequences. The company wants to identify content that is frequently watched together. Apply the FP-Growth algorithm to discover frequent viewing patterns. Construct an FP-Tree from the viewing data. Extract frequent itemsets efficiently without generating candidate sets. Generate association rules using support, confidence, and lift. Identify strong content relationships. Recommend movies and shows based on these patterns. Compare the efficiency of FP-Growth with Apriori. Visualize the strongest recommendations. Generate a detailed report summarizing the results. Write a program to implement this recommendation system.

Code:

```
!pip install -q mlxtend

import pandas as pd

import matplotlib.pyplot as plt

import time

from mlxtend.preprocessing import TransactionEncoder
```

```

from mlxtend.frequent_patterns import fpgrowth,apriori,association_rules

transactions=[

['Avengers','Iron Man','Spider-Man'],

['Avengers','Spider-Man'],

['Iron Man','Avengers'],

['Breaking Bad','Stranger Things'],

['Stranger Things','Wednesday'],

['Avengers','Iron Man','Spider-Man'],

['Breaking Bad','Money Heist'],

['Money Heist','Stranger Things'],

['Avengers','Spider-Man'],

['Wednesday','Stranger Things'],

['Iron Man','Spider-Man'],

['Breaking Bad','Money Heist','Stranger Things'],

['Avengers','Iron Man'],

['Money Heist','Wednesday'],

['Avengers','Iron Man','Spider-Man']

]

te=TransactionEncoder()

data=te.fit(transactions).transform(transactions)

df=pd.DataFrame(data,columns=te.columns_)

start=time.time()

frequent_itemsets=fpgrowth(df,min_support=0.2,use_colnames=True)

fp_time=time.time()-start

frequent_itemsets['Itemset']=frequent_itemsets['itemsets'].apply(lambda x: ', '.join(list(x)))

```

```

frequent_itemsets=frequent_itemsets.sort_values(by='support',ascending=False)

print("Frequent Viewing Patterns:")

print(frequent_itemsets[['Itemset','support']])

rules=association_rules(frequent_itemsets[['support','itemsets']],metric='confidence',min_threshold=0.5)

rules=rules[rules['lift']>1]

rules['Antecedents']=rules['antecedents'].apply(lambda x: ', '.join(list(x)))

rules['Consequents']=rules['consequents'].apply(lambda x: ', '.join(list(x)))

print("\nStrong Content Relationships:")

print(rules[['Antecedents','Consequents','support','confidence','lift']])

start=time.time()

apriori(df,min_support=0.2,use_colnames=True)

apriori_time=time.time()-start

print("\nFP-Growth Execution Time:",round(fp_time,6),"seconds")

print("Apriori Execution Time:",round(apriori_time,6),"seconds")

top_rules=rules.sort_values(by='lift',ascending=False).head(5)

labels=top_rules['Antecedents']+' → '+top_rules['Consequents']

plt.figure(figsize=(10,6))

plt.bar(labels,top_rules['lift'])

plt.xlabel('Content Recommendation Rules')

plt.ylabel('Lift Value')

plt.title('Strongest Streaming Content Recommendations Using FP-Growth')

plt.xticks(rotation=30,ha='right')

plt.tight_layout()

plt.show()

print("\nReport:")

```

```

print("FP-Growth was used to discover content that users frequently watch together.")

print("Unlike Apriori, FP-Growth uses an FP-Tree structure and does not generate large
numbers of candidate itemsets.")

print("Support measures how frequently content combinations occur.")

print("Confidence measures the probability of watching recommended content after watching
another item.")

print("Lift greater than 1 indicates a positive relationship between two content items.")

print("The strongest association rules can be used to recommend movies and TV shows to
users.")

print("FP-Growth is generally more efficient than Apriori for large datasets because it avoids
candidate generation.")

print("The streaming platform can use these patterns to provide personalized recommendations
and improve user engagement.")

```

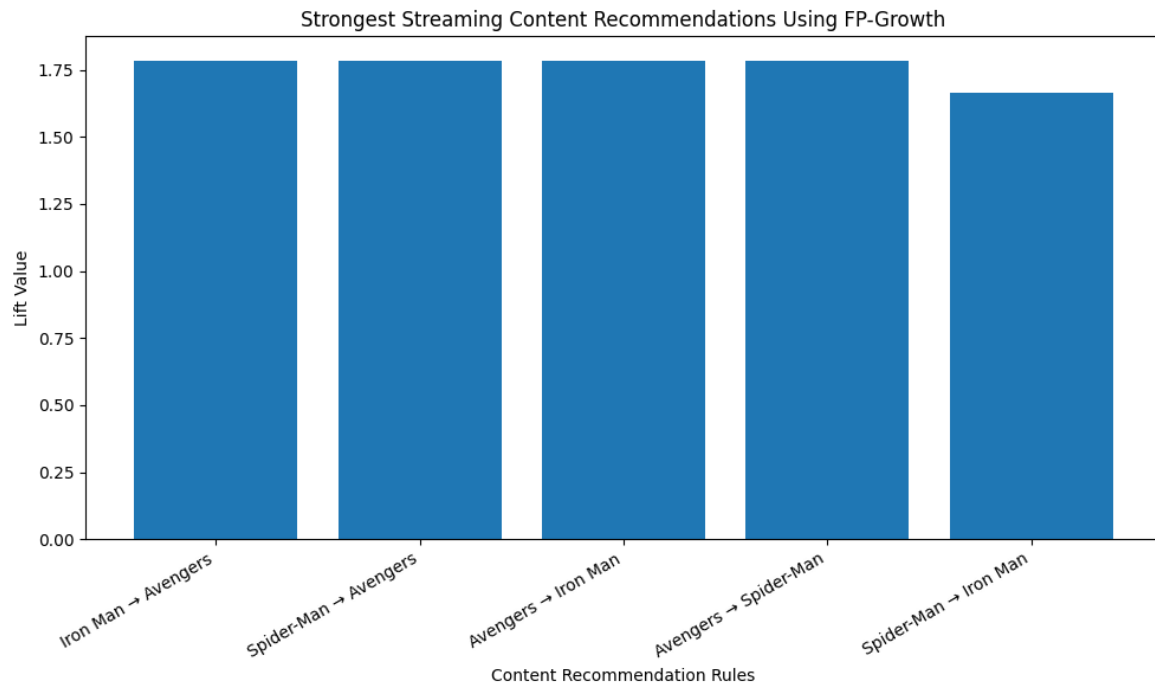
Output:

Frequent Viewing Patterns:

	Itemset	support
0	Avengers	0.466667
1	Iron Man	0.400000
2	Spider-Man	0.400000
3	Stranger Things	0.333333
7	Iron Man, Avengers	0.333333
8	Spider-Man, Avengers	0.333333
6	Money Heist	0.266667
9	Spider-Man, Iron Man	0.266667
4	Breaking Bad	0.200000
5	Wednesday	0.200000
10	Iron Man, Spider-Man, Avengers	0.200000

Strong Content Relationships:

	Antecedents	Consequents	support	confidence	lift
0	Iron Man	Avengers	0.333333	0.833333	1.785714
1	Avengers	Iron Man	0.333333	0.714286	1.785714
2	Spider-Man	Avengers	0.333333	0.833333	1.785714
3	Avengers	Spider-Man	0.333333	0.714286	1.785714
4	Spider-Man	Iron Man	0.266667	0.666667	1.666667
5	Iron Man	Spider-Man	0.266667	0.666667	1.666667
6	Spider-Man, Iron Man	Avengers	0.200000	0.750000	1.607143
7	Avengers, Iron Man	Spider-Man	0.200000	0.600000	1.500000
8	Spider-Man, Avengers	Iron Man	0.200000	0.600000	1.500000
9	Iron Man	Spider-Man, Avengers	0.200000	0.500000	1.500000
10	Spider-Man	Avengers, Iron Man	0.200000	0.500000	1.500000



31. Bank Customer Segmentation Using K-Means Clustering

A commercial bank has collected customer information including age, annual income, account balance, number of transactions, loan amount, and credit card usage. The marketing department wants to divide customers into meaningful groups for personalized banking services. Customers with similar financial behavior should belong to the same cluster. Before clustering, the dataset should be cleaned and standardized. Apply K-Means clustering using different values of K. Use the Elbow Method to identify the optimal number of clusters. Assign cluster labels to all customers. Visualize the clusters using suitable plots. Analyze the characteristics of each customer segment. Identify premium, regular, and low-engagement customers. Recommend targeted financial products for each cluster. Generate a detailed summary report. Write a program to perform this clustering task.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

from sklearn.decomposition import PCA

np.random.seed(42)
```

```
n=300

df=pd.DataFrame({

'Age':np.random.randint(18,70,n),

'AnnualIncome':np.random.randint(200000,2000000,n),

'AccountBalance':np.random.randint(10000,1000000,n),

'Transactions':np.random.randint(1,200,n),

'LoanAmount':np.random.randint(0,1000000,n),

'CreditCardUsage':np.random.randint(0,100,n)

})

df=df.dropna()

X=StandardScaler().fit_transform(df)

inertia=[]

for k in range(1,8):

    model=KMeans(n_clusters=k,random_state=42,n_init=10)

    model.fit(X)

    inertia.append(model.inertia_)

plt.figure(figsize=(8,5))

plt.plot(range(1,8),inertia,marker='o')

plt.xlabel('Number of Clusters (K)')

plt.ylabel('Inertia')

plt.title('Elbow Method for Bank Customer Segmentation')

plt.xticks(range(1,8))

plt.grid(True,alpha=0.3)

plt.show()

optimal_k=3
```



```

model=KMeans(n_clusters=optimal_k,random_state=42,n_init=10)

df['Cluster']=model.fit_predict(X)

cluster_summary=df.groupby('Cluster').mean().round(2)

print("Optimal Number of Clusters:",optimal_k)

print("\nCustomer Cluster Summary:")

print(cluster_summary)

cluster_names={}

sorted_clusters=cluster_summary['AccountBalance'].sort_values().index.tolist()

cluster_names[sorted_clusters[0]]='Low-Engagement'

cluster_names[sorted_clusters[1]]='Regular'

cluster_names[sorted_clusters[2]]='Premium'

df['CustomerSegment']=df['Cluster'].map(cluster_names)

print("\nCustomer Segments:")

print(df[['Age','AnnualIncome','AccountBalance','Transactions','LoanAmount','CreditCardUsage','CustomerSegment']].head(10))

print("\nNumber of Customers in Each Segment:")

print(df['CustomerSegment'].value_counts())

print("\nReport:")

print("The bank customer data was cleaned and standardized before applying K-Means clustering.")

print("The Elbow Method was used to analyze different values of K and select an appropriate number of customer clusters.")

print("Customers were divided into Premium, Regular, and Low-Engagement segments based on their financial characteristics.")

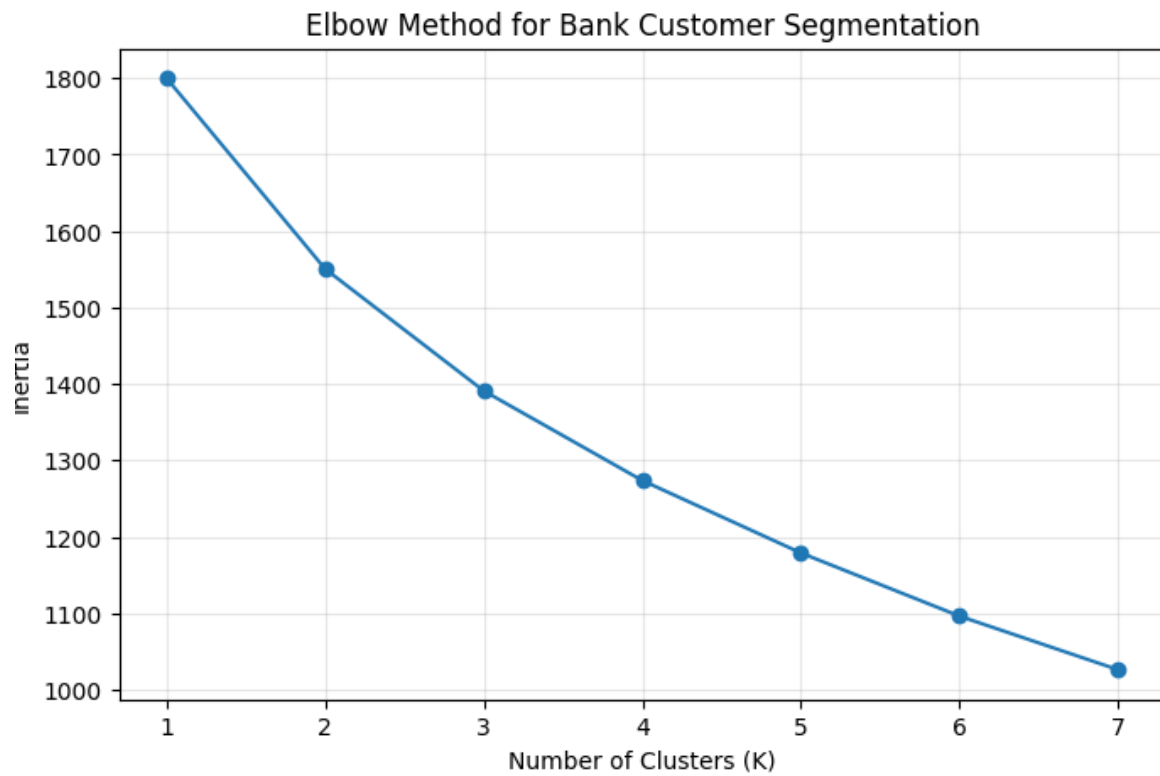
print("Premium customers can be offered wealth management, premium credit cards, and investment services.")

print("Regular customers can be offered personal loans, savings plans, and standard credit card products.")

```

```
print("Low-Engagement customers can be targeted with digital banking offers, rewards, and personalized promotions to increase engagement.")
```

Output:



```

-----
Cluster      Age  AnnualIncome  AccountBalance  Transactions  LoanAmount  \
0           37.27    1172679.55      675026.57        71.52    465767.16
1           45.49     723622.23      251104.85       117.13    548134.74
2           47.18    1378185.13      561522.22        93.64    458338.76

```

```

CreditCardUsage
Cluster
0           29.25
1           39.32
2           78.57

```

```

Customer Segments:
Age  AnnualIncome  AccountBalance  Transactions  LoanAmount  \
0   56      1540575      999873           33      951891
1   69      1441082      546226          118        8512
2   46      1190198      962699           65     656702
3   32       898376      363824          146     456292
4   60       469544       14000          177     108872
5   25       580002      735948           11     936720
6   38       898002      897474           85     772042
7   56      1788015      365754           26      93070
8   36       790978      261492           63     604055
9   40      1777363      900123           86     622117

```

```

CreditCardUsage  CustomerSegment
0                7      Premium
1               28      Regular
2               54      Premium
3                2  Low-Engagement
4               31  Low-Engagement
5                9      Premium
6               73      Regular
7               82      Regular
8               33  Low-Engagement
9               96      Regular

```

```

Number of Customers in Each Segment:
CustomerSegment
Regular      101
Premium      100
Low-Engagement  99
Name: count, dtype: int64

```

32. University Ranking Analysis Using Hierarchical Clustering

An education analytics company has collected information about universities including student strength, faculty-student ratio, research publications, placement percentage, infrastructure score, and international collaborations. The company wants to group universities with similar characteristics. Since the number of groups is unknown, Hierarchical Clustering should be

applied. Standardize the dataset before analysis. Compute the distance matrix among universities. Generate a dendrogram to visualize institutional similarities. Determine the appropriate number of clusters. Assign cluster labels to each university. Compare the characteristics of universities within each cluster. Identify top-performing and developing institutions. Provide recommendations for institutional improvement. Generate a final report. Write a program to complete this analysis.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import AgglomerativeClustering

from scipy.cluster.hierarchy import dendrogram, linkage

np.random.seed(42)

n=20

df=pd.DataFrame({

'University':['University '+str(i) for i in range(1,n+1)],

'StudentStrength':np.random.randint(2000,20000,n),

'FacultyStudentRatio':np.random.uniform(10,30,n),

'ResearchPublications':np.random.randint(100,2000,n),

'PlacementPercentage':np.random.uniform(50,100,n),

'InfrastructureScore':np.random.uniform(40,100,n),

'InternationalCollaborations':np.random.randint(5,100,n)

})

X=df.drop('University',axis=1)

X=StandardScaler().fit_transform(X)

distance_matrix=linkage(X,method='ward')

plt.figure(figsize=(12,6))
```

```

dendrogram(distance_matrix,labels=df['University'].values)

plt.title('Hierarchical Clustering Dendrogram of Universities')

plt.xlabel('Universities')

plt.ylabel('Distance')

plt.xticks(rotation=90)

plt.tight_layout()

plt.show()

optimal_k=3

model=AgglomerativeClustering(n_clusters=optimal_k,linkage='ward')

df['Cluster']=model.fit_predict(X)

cluster_summary=df.groupby('Cluster').mean(numeric_only=True).round(2)

print("Number of Clusters:",optimal_k)

print("\nUniversity Cluster Summary:")

print(cluster_summary)

performance=cluster_summary[['ResearchPublications','PlacementPercentage','Infrastructure
Score','InternationalCollaborations']].mean(axis=1)

sorted_clusters=performance.sort_values().index.tolist()

cluster_names={}

cluster_names[sorted_clusters[0]]='Developing Institutions'

cluster_names[sorted_clusters[1]]='Growing Institutions'

cluster_names[sorted_clusters[2]]='Top-Performing Institutions'

df['InstitutionCategory']=df['Cluster'].map(cluster_names)

print("\nUniversity Classification:")

print(df[['University','Cluster','InstitutionCategory']])

print("\nReport:")

print("The university dataset was standardized before applying Hierarchical Clustering.")

```

```
print("A distance matrix was computed using Ward linkage to measure similarities between universities.")
```

```
print("The dendrogram visualizes how similar institutions are grouped and helps determine an appropriate number of clusters.")
```

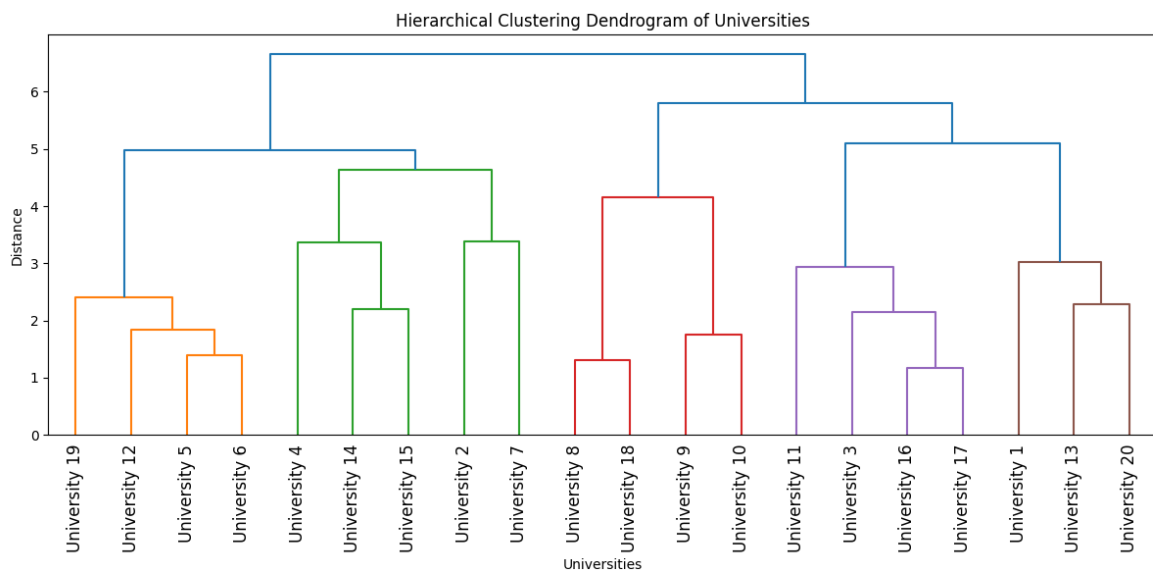
```
print("Universities were divided into three groups: Top-Performing, Growing, and Developing Institutions.")
```

```
print("Top-performing institutions generally show stronger research output, placement performance, infrastructure, and international collaborations.")
```

```
print("Developing institutions can improve by increasing research activities, strengthening industry partnerships, improving infrastructure, and expanding international collaborations.")
```

```
print("The clustering results can help education analysts compare similar universities and develop targeted institutional improvement strategies.")
```

Output:



Number of Clusters: 3

University Cluster Summary:

Cluster	StudentStrength	FacultyStudentRatio	ResearchPublications \
0	9950.14	22.16	1525.86
1	10479.44	14.74	1170.33
2	11158.00	21.54	494.75

Cluster	PlacementPercentage	InfrastructureScore	InternationalCollaborations
0	67.53	69.56	78.29
1	87.22	72.83	35.56
2	55.88	69.00	29.50

University Classification:

University	Cluster	InstitutionCategory
0 University 1	0	Top-Performing Institutions
1 University 2	1	Growing Institutions
2 University 3	0	Top-Performing Institutions
3 University 4	1	Growing Institutions
4 University 5	1	Growing Institutions
5 University 6	1	Growing Institutions
6 University 7	1	Growing Institutions
7 University 8	2	Developing Institutions
8 University 9	2	Developing Institutions
9 University 10	2	Developing Institutions
10 University 11	0	Top-Performing Institutions
11 University 12	1	Growing Institutions
12 University 13	0	Top-Performing Institutions
13 University 14	1	Growing Institutions
14 University 15	1	Growing Institutions
15 University 16	0	Top-Performing Institutions
16 University 17	0	Top-Performing Institutions
17 University 18	2	Developing Institutions
18 University 19	1	Growing Institutions
19 University 20	0	Top-Performing Institutions